

Введение

Matplotlib - это библиотека Python, используемая для создания диаграмм и графиков.

В этом первом уроке вы получите обзор основных команд, необходимых для построения и маркировки линейного графика. Понятия, которые вы изучите, включают:

- Создание линейного графика из данных
- Изменение внешнего вида линии
- Увеличение разных частей оси
- Нанесение меток на заголовки и оси
- Создание более сложного макета фигуры
- Добавление легенд к графикам
- Изменение меток и позиций галочки
- Сохранение того, что вы сделали

К концу урока вы станете мастером представления и организации данных на Python.

Прежде чем мы начнем работать с Matplotlib, нам нужно импортировать его в нашу среду Python.

```
from matplotlib import pyplot as plt
```

Это импортирует функции построения графиков из matplotlib и сделает их доступными с использованием более короткого имени plt. Большинство команд, которые мы изучим, будут начинаться с plt. идти вперед.

Базовый линейный график

Линейные графики полезны для визуализации того, как переменная изменяется с течением времени.

Некоторые возможные данные, которые будут отображаться в виде линейного графика:

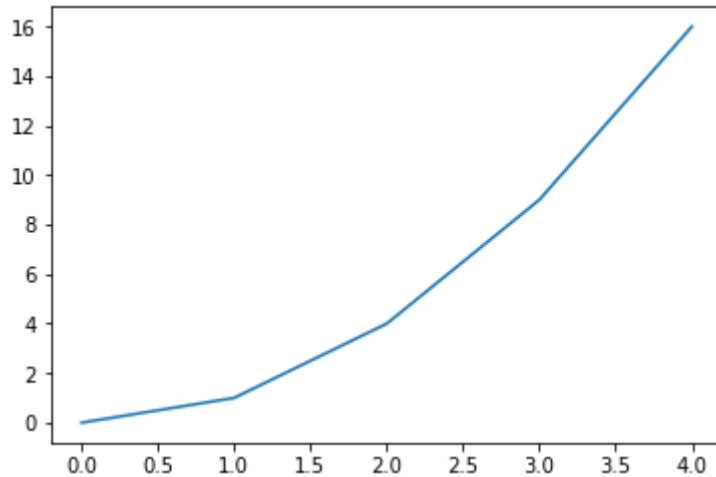
- средние цены на бензин за последнее десятилетие
- вес человека за последние пару месяцев
- средняя температура по долготе на разных широтах

Используя методы Matplotlib, следующий код создаст простой линейный график с помощью .plot () и отобразит его с помощью .show ():

```
x_values = [0, 1, 2, 3, 4]
y_values = [0, 1, 4, 9, 16]
plt.plot(x_values, y_values)
plt.show()
```

- x_values - это переменная, содержащая список значений x для каждой точки на нашем линейном графике.
- y_values - это переменная, содержащая список значений y для каждой точки на нашем линейном графике.

- plt - это имя, которое мы дали модулю Matplotlib, который мы импортировали в верхней части кода.
- plt.plot (x_values, y_values) создаст линейный график
- plt.show () фактически отобразит график



Задание

1. Мы собираемся построить простой график, показывающий, сколько кто-то тратил на обед за последнюю неделю. Сначала определите два списка days и money_spent, которые содержат следующие целые числа:

	Days	Money Spent
0		10
1		12
2		12
3		10
4		14
5		22
6		24

М

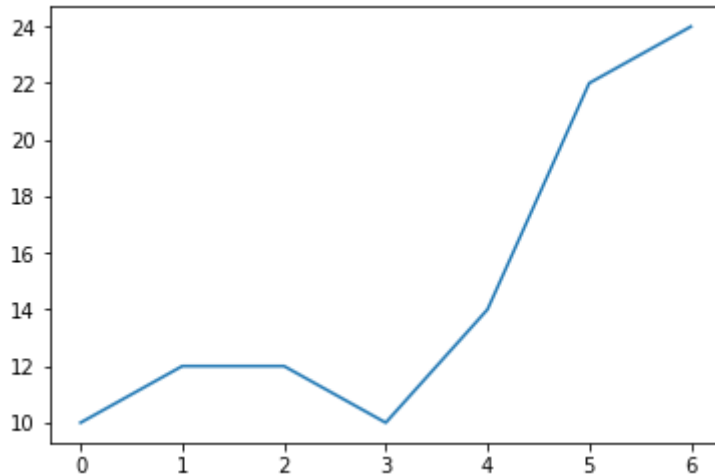
2. Постройте дни на оси x и money_spent на оси y с помощью plt.plot ().
3. Покажите график, используя plt.show ().

Базовая линия, участок II

Мы также можем отображать несколько линейных графиков на одном наборе осей. Это может быть очень полезно, если мы хотим сравнить два набора данных с одинаковыми категориями шкалы и оси.

Matplotlib автоматически поместит две линии на одни и те же оси и придаст им разные цвета, если вы вызовете `plt.plot()` дважды.

Давайте посмотрим на график, который мы построили в последнем упражнении для отслеживания расходов на обед, где дни указаны на оси x, а расходы (`money_spent`) - на оси y:

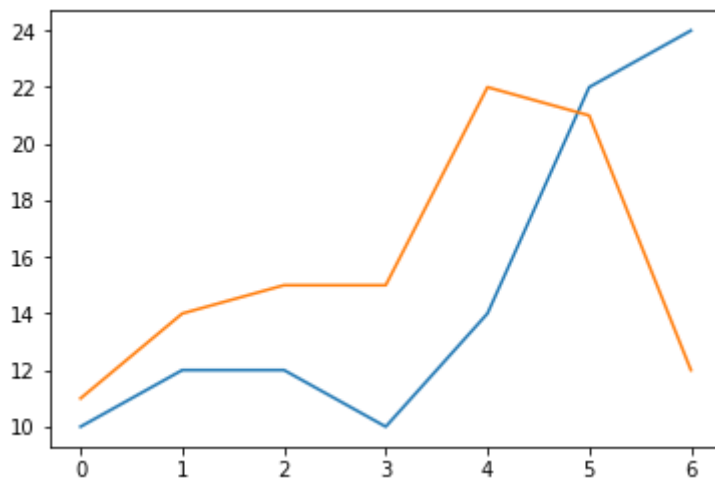


Мы могли бы добавить расходы на обед друга для сравнения следующим образом:

```
# Days of the week:
days = [0, 1, 2, 3, 4, 5, 6]
# Your Money:
money_spent = [10, 12, 12, 10, 14, 22, 24]
# Your Friend's Money:
money_spent_2 = [11, 14, 15, 15, 22, 21, 12]
# Plot your money:
plt.plot(days, money_spent)
# Plot your friend's money:
plt.plot(days, money_spent_2)
# Display the result:
plt.show()
```

W

Тогда мы получим две строки на одном и том же поле:



По умолчанию первая строка всегда синяя, а вторая всегда оранжевая. В следующем упражнении мы узнаем, как самостоятельно настроить эти линии.

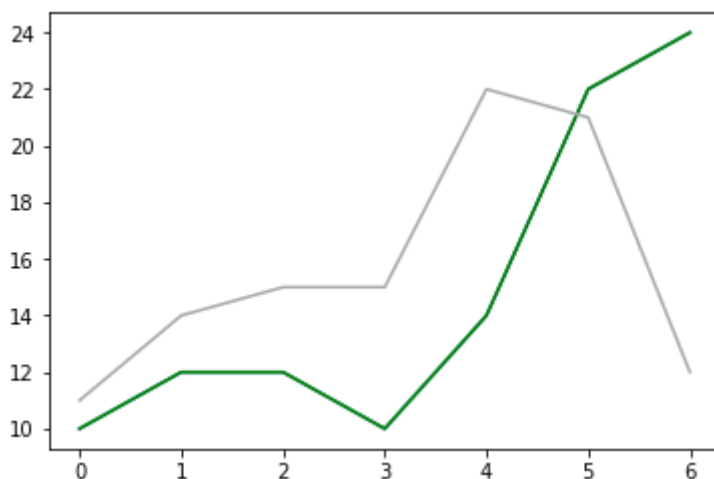
Задание

1. Мы определили списки, называемые временем, доходом и затратами. Постройте график зависимости дохода от времени.
2. График издержек в зависимости от времени на том поле
3. Покажите график.

Linestyles

Мы можем указать другой цвет для строки, используя ключевое слово `color` с именем цвета HTML или HEX-кодом:

```
plt.plot(days, money_spent, color='green')
plt.plot(days, money_spent_2, color='#AAAAAA')
```



Мы также можем сделать линию пунктирной или пунктирной, используя ключевое слово `linestyle`.

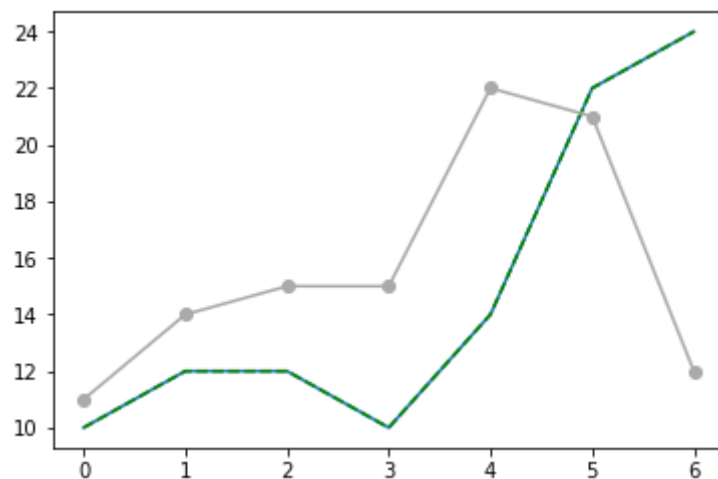
```
# Dashed:  
plt.plot(x_values, y_values, linestyle='--')  
# Dotted:  
plt.plot(x_values, y_values, linestyle=':')  
# No line:  
plt.plot(x_values, y_values, linestyle='')
```

Мы также можем добавить маркер, используя маркер ключевого слова:

```
# A circle:  
plt.plot(x_values, y_values, marker='o')  
# A square:  
plt.plot(x_values, y_values, marker='s')  
# A star:  
plt.plot(x_values, y_values, marker='*')
```

Чтобы увидеть все возможные варианты, ознакомьтесь с документацией Matplotlib. Вот пара значений, примененных к нашим графикам расходов на обед:

```
plt.plot(days, money_spent, color='green', linestyle='--')  
plt.plot(days, money_spent_2, color='#AAAAAA', marker='o')
```



Задание

1. Постройте график зависимости дохода от времени в виде фиолетовой («пурпурной») ('purple')пунктирной («--») линии.
2. Постройте график зависимости затрат от времени в виде линии с шестиугольным цветом # 82edc9 и квадратными маркерами ('s').

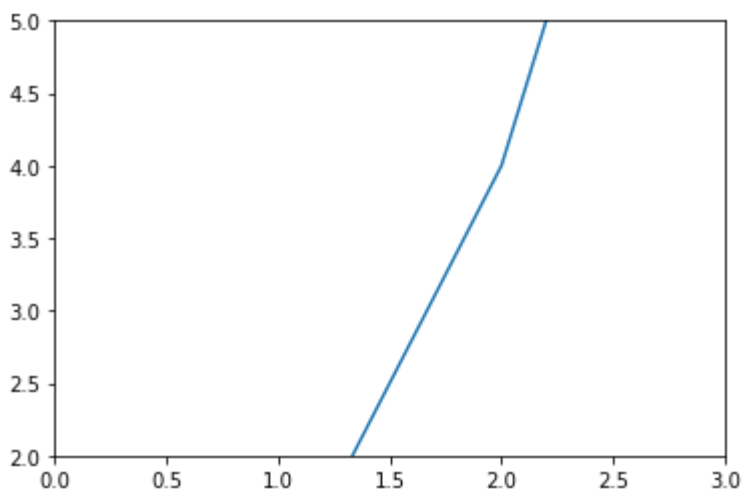
Оси и метки

Иногда может быть полезно увеличить или уменьшить масштаб графика, особенно если есть какие-то детали, которые мы хотим затронуть. Для увеличения мы можем использовать `plt.axis()`. Мы используем `plt.axis()`, передавая ему список в качестве входных данных. Этот список должен содержать:

- Минимальное отображаемое значение x
- Максимальное отображаемое значение x
- Отображаемое минимальное значение y
- Максимальное отображаемое значение y

Например, если мы хотим отобразить график от $x = 0$ до $x = 3$ и от $y = 2$ до $y = 5$, мы должны вызвать `plt.axis([0, 3, 2, 5])`.

```
x = [0, 1, 2, 3, 4]
y = [0, 1, 4, 9, 16]
plt.plot(x, y)
plt.axis([0, 3, 2, 5])
plt.show()
```



Задание

1. Мы построили линию, показывающую чьи-то расходы на кофе за последние 12 лет. Запустите код, чтобы увидеть получившийся график.

```
from matplotlib import pyplot as plt

x = range(12)
y = [3000, 3005, 3010, 2900, 2950, 3050, 3000, 3100, 2980, 2980, 2920, 3010]
plt.plot(x, y)
```

```
plt.show()
```

- Давайте изменим оси, чтобы увеличить масштаб нашей линейной диаграммы. Используйте `plt.axis()`, чтобы изменить оси так, чтобы ось `x` двигалась от 0 до 12, а ось `y` - от 2900 до 3100.

Маркировка осей

В конце концов, мы захотим показать эти графики другим людям, чтобы убедить их в важных тенденциях в наших данных. При этом мы хотим, чтобы наши сюжеты выглядели как можно более профессионально.

Первым шагом на пути к профессионально выглядящему графику является добавление меток к осям `x` и `y` и присвоение графику названия.

Мы можем пометить оси `x` и `y` с помощью `plt.xlabel()` и `plt.ylabel()`. Заголовок можно задать с помощью `plt.title()`.

Для всех этих команд требуется строка, которая представляет собой набор символов в одинарных (') или двойных (") кавычках.

```
"This is a string"
'This is also a string'

'This is NOT a string (the quotes do not match)'
```

Например, если кто-то отслеживал свое счастье (по 10-балльной шкале) в течение дня и хочет отобразить эту информацию с помеченными осями, мы можем использовать следующие команды:

```
hours = [9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20]
happiness = [9.8, 9.9, 9.2, 8.6, 8.3, 9.0, 8.7, 9.1, 7.0, 6.4, 6.9, 7.5]
plt.plot(hours, happiness)
plt.xlabel('Time of day')
plt.ylabel('Happiness Rating (out of 10)')
plt.title('My Self-Reported Happiness While Awake')
plt.show()
```

Это приведет к помеченному графику:



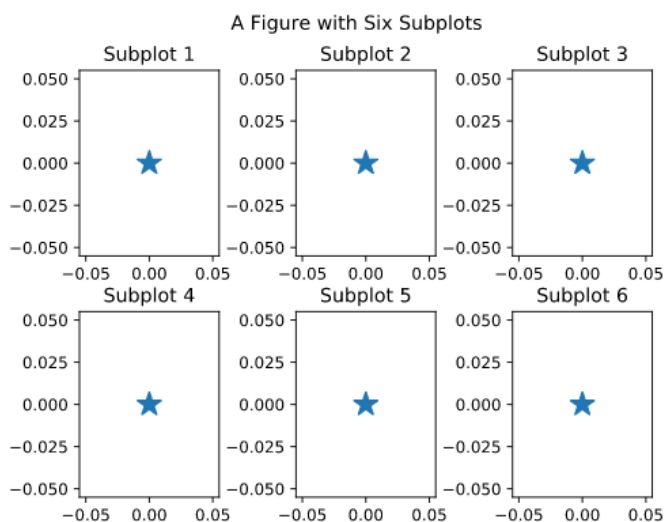
Задание

1. На предыдущем графике обозначьте ось абсцисс "Время".
2. Обозначьте ось Y "Доллары, потраченные на кофе".
3. Добавьте заголовок «Мои последние двенадцать лет употребления кофе».

Subplots

Иногда мы хотим отображать две линии рядом, а не в одном наборе осей x и y. Когда у нас есть несколько осей на одном изображении, мы называем каждый набор осей второстепенным графиком. Изображение или объект, содержащий все подграфики, называется фигурой.

У нас может быть много разных подграфиков на одном и том же рисунке, и мы можем расположить их разными способами. Мы можем думать о наших макетах как о рядах и столбцах подзаголовков. Например, на следующем рисунке есть шесть подзаголовков, разделенных на 2 строки и 3 столбца:



Мы можем создавать подграфики, используя `.subplot ()`.

Команде `plt.subplot ()` необходимо передать три аргумента:

- Количество рядов подграфиков
- Количество столбцов подграфики
- Индекс подграфикаа, который мы хотим создать

Например, команда `plt.subplot (2, 3, 4)` создаст «Subplot 4» из рисунка выше.

Любой `plt.plot ()`, идущий после `plt.subplot ()`, создаст линейный график в указанном подграфике. Например:

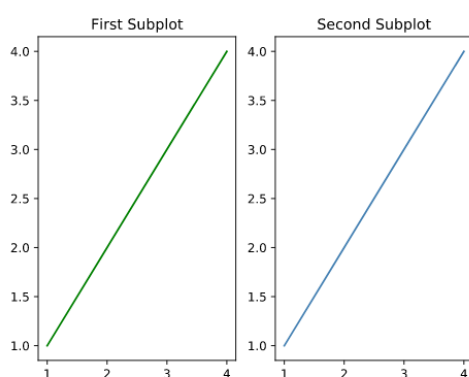
```
# Data sets
x = [1, 2, 3, 4]
y = [1, 2, 3, 4]

# First Subplot
plt.subplot(1, 2, 1)
plt.plot(x, y, color='green')
plt.title('First Subplot')

# Second Subplot
plt.subplot(1, 2, 2)
plt.plot(x, y, color='steelblue')
plt.title('Second Subplot')

# Display both subplots
plt.show()
```

В результате получится фигура с двумя графиками, расположенными следующим образом:



1. Мы определили для вас списки месяцев, температуры и `flight_to_hawaii`. Используя команду `plt.subplot`, постройте график зависимости температуры от месяцев в левом поле фигуры, состоящей из 1 строки с 2 столбцами.

```
from matplotlib import pyplot as plt
```

```
months = range(12)
temperature = [36, 36, 39, 52, 61, 72, 77, 75, 68, 57, 48, 48]
flights_to_hawaii = [1200, 1300, 1100, 1450, 850, 750, 400, 450, 400, 860, 990, 1000]
```

2. Постройте зависимости `flight_to_hawaii` от температуры на том же рисунке справа от вашего первого графика. Добавьте параметр «o» в конец вашего вызова к `plt.plot`, чтобы преобразовать график в диаграмму рассеяния, если хотите!
3. Покажите графики